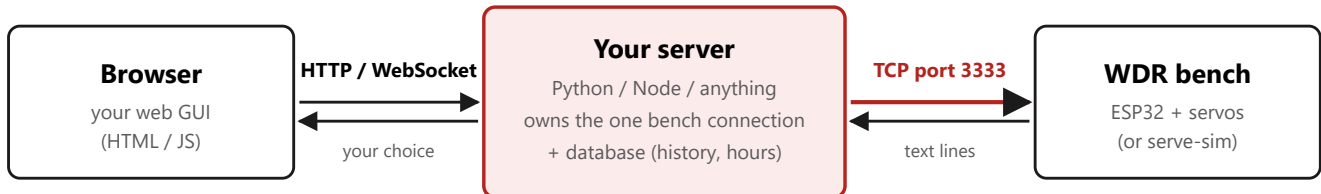


Connecting your controller to the bench

How your server talks to the WDR replay bench, with working code you can copy. Read this with the **WDR Manual and Protocol** PDF, which has every command.

1 The architecture



WHY A SERVER IN THE MIDDLE?

A web page can't open a TCP connection. Browsers only speak HTTP and WebSocket, so no JavaScript in a page can reach port 3333 directly. That's true on every computer and every browser. Your server opens the TCP connection and relays commands and replies to the page.

IT'S ALSO THE RIGHT DESIGN

History and hour meters must survive a closed browser tab, so they belong in your server's database. The bench accepts **one controller at a time**, so your server should hold that single connection and share it with every browser tab.

2 Getting started

STEP	WHAT TO DO
Install	Python 3, then <code>pip install pyserial</code> . You get <code>wdr_tool.py</code> , <code>bench_client.py</code> and <code>bench_client.js</code> from Welkinrim
Fake bench on your laptop	<code>python wdr_tool.py serve-sim</code> . It listens on <code>127.0.0.1:3333</code> and behaves exactly like the real bench. Leave it running
Poke it by hand	<code>python wdr_tool.py term --host 127.0.0.1</code> , then type <code>PING</code> , <code>INFO</code> , <code>STATUS</code>
Run the example client	<code>python bench_client.py 127.0.0.1</code> or <code>node bench_client.js 127.0.0.1</code> . It uploads, plays, reads telemetry and counters
Build your server on it	Copy the Bench class into your server (next page)
Real bench	Join the event Wi-Fi. Welkinrim gives you the bench IP. Swap <code>127.0.0.1</code> for it and nothing else changes. Book bench time: only one team can be connected at once

3 The client code (Python, from `bench_client.py`)

This is the whole connection layer. The Node.js version in `bench_client.js` works the same way. Both were tested on the real bench.

```
# standard library only
import queue, socket, threading

class Bench:
    def __init__(self, host, port=3333):
        self.sock = socket.create_connection((host, port), timeout=5)
        self.replies = queue.Queue()      # OK / ERR lines, in order
        self.lock = threading.Lock()     # one command in flight at a time
        threading.Thread(target=self._reader, daemon=True).start()

    def _reader(self):
        # Every line from the bench is routed by its FIRST WORD.
        f = self.sock.makefile("r", encoding="ascii", newline="\n")
        for line in f:
            line = line.strip()
            if line.startswith(("OK", "ERR")):
                self.replies.put(line)
            elif line.startswith("TEL"):
                self.on_tel(line)        # live status, every 0.5 s
            elif line.startswith("EVT"):
                self.on_evt(line)        # EVT CYCLE n, EVT DONE
            self.replies.put(None)       # connection closed

    def cmd(self, line, timeout=2.0):
        """Send one command and wait for its one reply."""
        with self.lock:
            self.sock.sendall((line + "\n").encode("ascii"))
            return self.replies.get(timeout=timeout)

    def on_tel(self, line): ...          # push to the browser
    def on_evt(self, line): ...          # write to your database

def kv(reply):  # 'OK STATUS state=RUNNING cycle=3' -> {'state': 'RUNNING', 'cycle': '3'}
    return dict(t.split("=", 1) for t in reply.split()[2:] if "=" in t)

# usage
b = Bench("127.0.0.1")
b.cmd("TEL 1")
s = kv(b.cmd("STATUS"))
print(s["state"], s["cycle"], "of", s["target"])
```

4 The three rules this code gets right

- 1 Route every line by its first word.** TEL and EVT lines can arrive *between* a command and its reply. If you just read "the next line" after sending, your replies drift out of step.
- 2 One command at a time.** Send, wait for its OK / ERR, then send the next. The lock makes that safe when several browser requests arrive at once.
- 3 Plan for the connection dropping.** When `cmd()` returns `None` (connection closed) or raises `queue.Empty` (no reply in time), reconnect. Then send `TEL 1` again (telemetry is off for every new connection) and `STATUS` to catch up. The bench kept playing and counting while you were gone.

5 GUI feature → bench commands

FEATURE	COMMANDS	NOTES
Connection badge	PING every few seconds, INFO	INFO up= is seconds since boot. If it drops, the bench rebooted
Import a flight	LOAD → F × n → COMMIT	Check SUM. See section 6
Start / Pause / Stop	START [cycles], PAUSE, RESUME, STOP	ERR STATE means the button isn't valid right now. Grey it out
Cycle target	START 100	EVT DONE when reached, and the bench stops itself
Manual servo sliders	SET <ch> <us>	Only while STOPPED, 500 to 2500 μs
Live dashboard	TEL 1, then TEL lines	state, cycle, frame, μs per channel. Progress = frame ÷ frames
Hour meters	COUNTERS	Lifetime cycles, run_s, active_s per channel. They survive power cuts
History	EVT lines + your own log	Timestamp every event and state change in your server and store it in your database

6 Uploading real flights

- **Resample** your log to evenly spaced frames at one rate (10 to 100 frames/s). Flight logs have uneven timestamps.
- **Pick the channels** the user wants: at most ch from INFO (4 on this bench).
- **Convert or clip values** to 500...2500 μs. Anything outside gets ERR RANGE and aborts the upload.
- **Mind the size limit:** maxframes from INFO, 8000 on this bench. That's 160 s at 50 frames/s or 13 min at 10/s. Choose a lower rate or a section of the flight.
- **Show a progress bar.** Each F line is a round trip, about 13 ms over Wi-Fi, so 4000 frames take about a minute.
- The profile is saved in the bench's flash and survives a power cut.

7 Good to know

SITUATION	WHAT HAPPENS / WHAT TO DO
Browser can't reach the bench	Expected: browsers can't do TCP. Your server must connect (section 1)
My connection closed by itself	Someone else connected: one controller at a time. Only your server should connect, and don't also run wdr_tool.py term against the bench
Telemetry stopped after a reconnect	Normal. Send TEL 1 again
START right after a big upload is slow	The bench finishes saving the profile to flash first (up to about a second)
Rates above 50 frames/s	Servos take one pulse every 20 ms, so the bench plays at most 50 distinct frames per second
Can't connect at all	Same Wi-Fi network as the bench? Right IP? Campus Wi-Fi often blocks device-to-device traffic, so use the event network